

RegRadar UI

Component Library · Design System · Site Architecture · Angular Implementation

For: Frontend Developer · Stack: Angular 17+ · May 2026

Note for the Frontend Developer: This document is your complete UI reference. It covers the design system, all pages and their components, site inspiration sources, Angular-specific implementation notes, and hosting. Read it top to bottom once, then use individual sections as a build reference as you go.

■ 1. Site Overview & Inspirations

RegRadar has four distinct sections, each with a different purpose and audience. Each section takes its design inspiration from a best-in-class reference site.

Section	URL	Audience	Inspiration	Tone
Marketing Home	/	Prospects, visitors	Linear.app	Dark, editorial, bold
Features Page	/features	Evaluating developers	Linear.app	Clean, structured, confident
Developer Dashboard	/dashboard	Paying customers	Vercel Dashboard	Functional, data-dense, minimal
API Playground	/playground	Onboarding devs	ReadMe.io / Hoppscotch	Dark, interactive, focused
Docs Site	/docs	Active developers	Stripe Docs	White, clear hierarchy, searchable
Pricing Page	/pricing	Decision makers	Linear.app	Clean, trust-building, clear CTAs

1.1 Inspiration Breakdown

Linear.app

linear.app · Used for: Home page + Features page

- Dark background (#0A0A0A) with tight, editorial typography
- Section-by-section scroll with large feature callouts
- Minimal navigation — logo, 3-4 links, one CTA button
- Animated gradient glows behind key hero elements
- Feature sections alternate: text left / visual right, then flip
- Monochrome with one sharp accent color (theirs is purple — ours is blue #2563EB)
- Copy is confident and short — no paragraph walls

Vercel Dashboard

vercel.com/dashboard · Used for: Developer Dashboard (post-login)

- Sidebar navigation — fixed left, icon + label, collapsible
- Top bar: breadcrumb + user avatar + notification bell
- Card-based layout — each metric or feed item in a bordered card
- Monospace numbers for API call counts, latency, error rates
- Status indicators: green dot (live), yellow (warning), red (error)
- Table rows for subscription list — sortable, filterable inline
- Dark theme default with a clean light mode option

Hoppscotch / ReadMe.io

hoppscotch.io · Used for: API Playground

- Three-panel layout: left (request builder), center (params), right (response)
- Syntax-highlighted JSON response viewer with copy button
- Endpoint dropdown with color-coded method badges (POST=blue, GET=green, DELETE=red)
- Send button is the hero action — large, prominent
- Environment variable support (base URL, API key pre-filled from dashboard)
- Request history sidebar — last 10 calls with timestamps
- Dark-only — no light mode needed here

Stripe Docs

stripe.com/docs · Used for: Documentation Site

- White background, clean two-column layout: sidebar nav + content
- Sticky sidebar with collapsible sections per API resource
- Code blocks with language tabs (Python / TypeScript / curl)
- Right-side response preview panel on API reference pages
- Search bar prominent at top — keyboard shortcut (Cmd+K)
- Breadcrumb trail on every page
- Light — contrasts with the dark dashboard and playground

■ 2. Design System

2.1 Typography

Two fonts only. Load both from Google Fonts. Never mix in system fonts.

Role	Font	Weights	Used For
Display / Heading	Geist (Vercel)	600, 700	All headings, nav links, hero text, page titles
Body / UI	Geist Mono	400, 500	All body text, labels, table content, API values
Code blocks	Geist Mono	400	All code snippets, JSON, endpoint URLs

Angular: Add to styles.scss → `@import url('https://fonts.googleapis.com/css2?family=Geist:wght@400;500;600;700&family=Geist+Mono:wght@400;500&display;=swap');`

2.2 Colour Palette

Define all colours as CSS custom properties in `:root`. Never hardcode hex values in component CSS.

	Surface / BG	#07090F	Page background (dark sections)
	Surface Raised	#0F172A	Card backgrounds, sidebar
	Surface Border	#1E293B	Card borders, dividers
	Primary	#2563EB	CTAs, active states, links
	Accent	#38BDF8	Highlights, badges, hover glows
	Success / Green	#10B981	Severity: minor, live status dots
	Warning / Yellow	#F59E0B	Severity: major, pending states
	Danger / Red	#EF4444	Severity: critical, error states
	Purple	#8B5CF6	Webhook delivery status, special badges
	Text Primary	#F8FAFC	Main body text on dark backgrounds
	Text Secondary	#94A3B8	Labels, captions, placeholder text

```
/* styles.scss – define once, use everywhere */
:root {
  --bg: #07090F;
  --bg-raised: #0F172A;
  --border: #1E293B;
  --primary: #2563EB;
  --accent: #38BDF8;
  --success: #10B981;
  --warning: #F59E0B;
  --danger: #EF4444;
  --text: #F8FAFC;
```

```
--text-muted: #94A3B8;  
}
```

2.3 Spacing System

Use a strict 4px base grid. All padding, margin, and gap values must be multiples of 4.

Token	Value	Use Case
--space-1	4px	Icon internal padding, tiny gaps
--space-2	8px	Inline element spacing, tight badges
--space-3	12px	List item padding, small card padding
--space-4	16px	Standard card padding, form field gaps
--space-5	24px	Section inner spacing, large card padding
--space-6	32px	Section gaps, large component spacing
--space-8	48px	Page-level section spacing
--space-10	64px	Hero section vertical rhythm

■ 3. Page-by-Page Component Breakdown

3.1 Home Page (/)

Inspiration: Linear.app · Theme: Dark · Goal: Convert visitors to sign-ups

Navbar

Inspired by: Linear.app

Top navigation bar. Fixed position, blurred glass background (backdrop-filter: blur(12px)). Logo left. Links centre: Features, Pricing, Docs, Changelog. CTA right: 'Get API Key' button. Collapses to hamburger on mobile.

- Logo: 'RegRadar' in Geist 600, white + radar icon SVG
- Nav links: 14px, text-muted colour, hover → text-primary
- CTA button: solid primary (#2563EB), 12px padding, 6px border-radius
- Background: rgba(7,9,15,0.8) + backdrop blur on scroll

Hero Section

Inspired by: Linear.app

Full-viewport-height section. Large headline, sub-headline, two CTAs, animated background glow. This is the most important section — it must stop the scroll.

- Headline: 52–64px Geist 700, white, max 8 words — 'Regulatory changes. Delivered instantly.'
- Sub: 18px, text-muted, 2 lines max — explain what it does simply
- CTAs: Primary 'Start Free' + Ghost 'View Docs' side by side
- Background: radial gradient glow in primary blue, very subtle, centred behind text
- Optional: subtle animated grid pattern overlay (CSS only, low opacity)

Logos / Social Proof Bar

Inspired by: Linear.app

A single row of greyscale company logos with label 'Trusted by teams at...' Builds immediate credibility.

- 6–8 placeholder logos (use generic tech company SVGs initially)
- Greyscale filter, hover removes filter
- Slow horizontal marquee animation on mobile

Feature Highlights (x3 sections)

Inspired by: Linear.app

Three alternating text+visual sections. Each highlights one core value prop. Scroll-triggered fade-in animation.

- Section 1: Webhooks — text left, animated JSON payload card right
- Section 2: AI Summaries — text right, severity badge + summary card left
- Section 3: Coverage — text left, jurisdiction map/grid visual right
- Visual side: dark card with subtle border, code or data UI mockup inside

Footer

Inspired by: Linear.app

Simple 4-column footer. Dark background matching page.

- Col 1: Logo + one-line description
- Col 2: Product links (Features, Pricing, Changelog)
- Col 3: Developer links (Docs, API Reference, SDKs, Status)
- Col 4: Company links (About, Blog, Contact)
- Bottom row: © 2026 RegRadar · Privacy · Terms

3.2 Features Page (/features)

Inspiration: Linear.app · Theme: Dark · Goal: Show depth, earn trust from technical evaluators

Feature Grid Hero

Inspired by: Linear.app

Page opens with a short hero (no full viewport height) — just headline + sub, then immediately flows into the feature grid below.

- Headline: 'Everything you need to monitor compliance' — 36px
- Sub: 2-line description, 16px text-muted

Feature Card Grid

Inspired by: Linear.app

3-column grid of feature cards. Each card has an icon, title, and 2-line description. Cards have a subtle border and hover lift effect.

- Icon: 24px SVG in accent colour on a dark raised card
- Title: 14px Geist 600
- Description: 13px text-muted, 2 lines
- Hover: border colour brightens to primary, box-shadow lifts
- ~9–12 cards total covering: Webhooks, AI Summaries, Structured Diff, Severity Scoring, Source Verification, Historical Archive, Multi-jurisdiction, Retry Logic, SDKs

Deep-Dive Sections

Inspired by: Linear.app

After the grid, 3–4 expanded sections go deep on the most important features. Each has a large visual (code block, JSON example, or diagram) with explanation text.

- Webhook Delivery — show the full payload structure with syntax highlighting
- Severity Scoring — show the three levels with colour badges and real examples
- Structured Diff — before/after code comparison side by side

CTA Banner

Inspired by: Linear.app

End of page: full-width dark banner with headline + single CTA button.

- 'Start monitoring in 5 minutes' — primary button → /dashboard/signup

3.3 Developer Dashboard (/dashboard)

Inspiration: Vercel Dashboard · Theme: Dark · Goal: Give developers control and visibility

App Shell / Layout

Inspired by: Vercel Dashboard

Fixed sidebar left + top bar + scrollable main content area. This is the persistent shell wrapping every dashboard page.

- Sidebar width: 220px (expanded) / 56px (collapsed)
- Sidebar items: Overview, Subscriptions, Change Feed, API Keys, Usage, Settings
- Active item: primary background, white text, left border accent
- Top bar: page title (breadcrumb) + notification bell + user avatar dropdown
- Main content: max-width 1200px, centred, 32px horizontal padding

Overview / Home

Inspired by: Vercel Dashboard

Landing page after login. Shows key metrics at a glance + recent changes.

- 4 stat cards top row: Total Subscriptions, Changes Today, Webhooks Delivered, API Calls (30d)
- Stat cards: large number in monospace, label below, subtle trend arrow
- Recent Changes table: last 10 changes, columns: Jurisdiction · Industry · Severity · Summary · Time
- Severity column: coloured badge — red=critical, yellow=major, green=minor
- Webhook Health chart: 7-day delivery success/failure bar chart (Recharts equivalent for Angular: ngx-charts)

Subscriptions Page

Inspired by: Vercel Dashboard

Full list of all subscriptions with create/edit/delete actions.

- Create button: top right, primary, opens a slide-over panel (not a modal)
- Table columns: Name · Jurisdiction · Industry · Topics · Status · Webhook URL · Created
- Status toggle: active/paused — toggle switch, changes via PATCH API call
- Row actions: Edit (pencil icon) · Delete (trash icon, requires confirmation)
- Empty state: centred illustration + 'No subscriptions yet' + Create CTA

Change Feed Page

Inspired by: Vercel Dashboard

Paginated, filterable list of all detected regulatory changes.

- Filters bar: Jurisdiction dropdown · Industry dropdown · Severity multi-select · Date range picker
- Each row expands on click to show full summary + diff viewer
- Diff viewer: two-column before/after with added lines in green, removed in red
- Pagination: cursor-based, 'Load more' button at bottom
- Export button: download current filter as JSON or CSV

API Keys Page

Inspired by: Vercel Dashboard

Create, view, revoke API keys.

- Keys are shown masked (sk-rr-.....1234) with a 'Reveal' button
- Create key: enter a name, click Generate — key shown once in a highlighted box with copy button
- Revoke: red button, requires typing key name to confirm (like Vercel does for project deletion)
- Last used timestamp shown for each key

Usage Page

Inspired by: Vercel Dashboard

API usage metrics and billing overview.

- Line chart: API calls per day for last 30 days
- Bar chart: Webhook delivery success vs failure per day
- Stat cards: Current period calls · Calls remaining · Reset date
- Current plan badge + Upgrade CTA if on Free tier
- Usage breakdown table: endpoint × count × avg latency

3.4 API Playground (/playground)

Inspiration: Hoppscotch · Theme: Dark only · Goal: Let developers try the API without writing code

The playground is the single most important onboarding tool. A developer who successfully fires a request and sees a real webhook payload in the playground will convert to a paying customer at a dramatically higher rate. Invest time here.

Three-Panel Layout

Inspired by: Hoppscotch

The playground uses a three-panel layout that fills the full browser viewport. No sidebar navigation here — this is a focused tool.

- Left panel (280px): Endpoint list, grouped by resource. Click to load into centre panel.
- Centre panel (flexible): Request builder — method badge, URL, headers, request body editor
- Right panel (400px): Response viewer — status code, latency, syntax-highlighted JSON
- Top bar: RegRadar logo + 'Back to Dashboard' link + API key selector dropdown

Endpoint List (Left Panel)

Inspired by: Hoppscotch

Grouped list of all API endpoints. Click any to pre-populate the request builder.

- Group headers: SUBSCRIPTIONS / CHANGES / SEARCH / WEBHOOKS
- Each item: METHOD badge (POST=blue, GET=green, DELETE=red, PATCH=yellow) + endpoint path
- Active item highlighted with left border accent
- Saved requests section at bottom (localStorage)

Request Builder (Centre Panel)

Inspired by: Hoppscotch

The main interaction area. Build and send API requests.

- URL bar: full endpoint URL, editable, with base URL pre-filled from environment
- Method badge: colour-coded, clickable to change method
- Tabs below URL bar: Params · Headers · Body · Auth
- Body tab: Monaco-style JSON editor with syntax highlighting (use ngx-monaco-editor)
- Auth tab: API key pre-filled from dashboard session — developer just hits Send
- Send button: large, primary blue, full width at bottom of centre panel
- Environment selector: dropdown to switch between Local / Staging / Production

Response Viewer (Right Panel)

Inspired by: Hoppscotch

Shows the API response after sending a request.

- Status line: HTTP status code badge (200=green, 4xx=yellow, 5xx=red) + latency in ms
- Response tabs: Body · Headers · Timeline
- Body: syntax-highlighted JSON with line numbers, collapsible nodes
- Copy button: copies raw JSON to clipboard
- Response size indicator: shown in KB
- Error state: red border, error message in plain English

Webhook Simulator

Inspired by: Hoppscotch

Special panel in the playground — lets developers test their webhook endpoint by firing a sample payload to any URL they enter.

- Input: webhook URL field
- Payload selector: dropdown of sample payloads (Critical KYC change, Minor guideline update, etc.)
- Fire button: sends the payload, shows the response from their server
- Delivery log: shows status code, latency, response body from their endpoint
- Signature: shows the HMAC-SHA256 signature so they can test verification

■ ■ 4. Angular Implementation Notes

4.1 Angular Version & Setup

```
# Use Angular 17+ with standalone components
ng new reg radar-frontend --routing --style=scss --standalone

# Key packages to install
npm install @angular/cdk # Component Dev Kit (overlays, drag-drop)
npm install ngx-charts # Charts for dashboard (Recharts equivalent)
npm install ngx-monaco-editor # Code editor for playground
npm install @angular/animations # For page transitions and micro-animations
npm install lucide-angular # Icon library (matches Linear aesthetic)
npm install tailwindcss # Utility CSS
```

4.2 Project Structure

```
src/
  ■ ■ ■ app/
    ■ ■ ■ ■ core/
      ■ ■ ■ ■ ■ services/
        ■ ■ ■ ■ ■ api.service.ts # All HTTP calls to RegRadar backend
        ■ ■ ■ ■ ■ auth.service.ts # API key management
        ■ ■ ■ ■ ■ webhook.service.ts # Webhook simulator logic
      ■ ■ ■ ■ ■ interceptors/
        ■ ■ ■ ■ ■ auth.interceptor.ts # Attach API key to every request
      ■ ■ ■ ■ ■ shared/
      ■ ■ ■ ■ ■ components/
      ■ ■ ■ ■ ■ navbar/
      ■ ■ ■ ■ ■ severity-badge/ # Critical / Major / Minor badge
      ■ ■ ■ ■ ■ code-block/ # Syntax highlighted code viewer
      ■ ■ ■ ■ ■ status-dot/ # Live / paused indicator dot
      ■ ■ ■ ■ ■ empty-state/ # Reusable empty state component
      ■ ■ ■ ■ ■ pipes/
        ■ ■ ■ ■ ■ timeago.pipe.ts # '3 minutes ago' formatting
      ■ ■ ■ ■ ■ pages/
        ■ ■ ■ ■ ■ home/
        ■ ■ ■ ■ ■ features/
        ■ ■ ■ ■ ■ pricing/
        ■ ■ ■ ■ ■ dashboard/
          ■ ■ ■ ■ ■ overview/
          ■ ■ ■ ■ ■ subscriptions/
          ■ ■ ■ ■ ■ change-feed/
          ■ ■ ■ ■ ■ api-keys/
          ■ ■ ■ ■ ■ usage/
        ■ ■ ■ ■ ■ playground/
      ■ ■ ■ ■ ■ app.routes.ts
    ■ ■ ■ ■ ■ styles/
      ■ ■ ■ ■ ■ _variables.scss # All CSS custom properties
```

```
■■■ _typography.scss
■■■ _components.scss
■■■ styles.scss # Global styles + imports
```

4.3 Key Angular Patterns to Use

Standalone Components: Use standalone: true on every component — no NgModules. Cleaner, faster, Angular 17+ default.

Signals for State: Use Angular Signals (signal(), computed(), effect()) instead of RxJS where possible. Simpler reactive state for dashboard data.

HTTP Client: Use inject(HttpClient) in services. Add the auth interceptor to attach the API key header automatically.

Route Guards: Protect /dashboard/** routes with an AuthGuard that checks for a valid API key in localStorage.

Lazy Loading: Lazy-load the Dashboard module and Playground module — they're heavy and not needed on the marketing pages.

CDK Overlay: Use Angular CDK for the slide-over panels (subscription create/edit). Do not use a third-party modal library.

OnPush Change Detection: Use ChangeDetectionStrategy.OnPush on all dashboard components for performance.

4.4 API Integration

```
// core/services/api.service.ts
import { inject, Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { environment } from '../../../environments/environment';

@Injectable({ providedIn: 'root' })
export class ApiService {
  private http = inject(HttpClient);
  private base = environment.apiUrl; // 'http://localhost:8000/v1'

  getSubscriptions() {
    return this.http.get(`${this.base}/subscriptions`);
  }

  createSubscription(data: CreateSubscriptionDto) {
    return this.http.post(`${this.base}/subscriptions`, data);
  }

  getChanges(params?: ChangeQueryParams) {
    return this.http.get(`${this.base}/changes`, { params });
  }
}
```

■ 5. Hosting & Deployment

5.1 Recommended Hosting: Vercel

The frontend is a Next.js-equivalent Angular app. The best hosting option for Angular in 2026 is Vercel (via static output) or Netlify. Vercel is recommended because it matches your dashboard inspiration and has the best DX.

Platform	Cost	Angular Support	Best For	Recommended
Vercel	Free tier	Excellent	Production + preview deploys	■ Yes
Netlify	Free tier	Excellent	Production deploys	■ Alternative
AWS S3+CF	~\$2–5/month	Manual setup	Full AWS stack integration	Phase 2+
Firebase	Free tier	Good	If using Firebase Auth/DB	Only if needed

5.2 Vercel Deployment Steps

```
# 1. Build Angular for static output
ng build --configuration production

# 2. vercel.json in project root
{
  "buildCommand": "ng build --configuration production",
  "outputDirectory": "dist/regradar-frontend/browser",
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
}

# 3. Environment variables in Vercel dashboard
VITE_API_URL = https://api.regradar.dev

# 4. Deploy
vercel --prod
```

5.3 Preview Deployments

Connect the GitHub repo to Vercel. Every pull request automatically gets a unique preview URL (e.g. regradar-git-feature-subscriptions-abc.vercel.app). This means your backend teammate can review UI changes without running the frontend locally — huge time saver.

5.4 Environment Configuration

```
// src/environments/environment.ts (local dev)
export const environment = {
  production: false,
  apiUrl: 'http://localhost:8000/v1'
};

// src/environments/environment.production.ts
export const environment = {
  production: true,
  apiUrl: 'https://api.regradar.dev/v1'
};
```

■ 6. Frontend Developer Checklist — Phase 1

Start here. In this order.

Day 1

- Set up Angular 17 project with Tailwind + SCSS variables
- Define all colour tokens in `_variables.scss`
- Install Geist font from Google Fonts
- Create AppShell component (sidebar + top bar) — no logic yet, just layout
- Set up routing: `/` → home, `/dashboard` → shell, `/playground` → playground

Day 2–3

- Build Navbar and Footer (marketing pages)
- Build Home page Hero section (static, no API)
- Build the Subscriptions page inside dashboard shell
- Wire POST `/subscriptions` to the Create Subscription form
- Wire GET `/subscriptions` to the subscriptions table

Day 4–5

- Build the Change Feed page with filters
- Build API Keys page (create + revoke)
- Build Overview page with 4 stat cards (wire to real data if backend ready, mock if not)
- Add SeverityBadge shared component (reuse across all pages)
- Basic playground: URL bar + Send button + response viewer

Week 2+

- Webhook simulator in playground
- Usage charts (ngx-charts)
- Animations: page transitions, card hover effects
- Mobile responsive: sidebar collapses, tables scroll horizontally
- Deploy to Vercel, connect to GitHub

Remember: The backend developer (API + scraping) and frontend developer (Angular) should agree on the API contract on Day 1 and then work independently. The backend exposes GET `/subscriptions` and POST `/subscriptions` by end of Day 1. The frontend mocks the data until then. Don't block each other.